

Distributed Data

What breaks when one machine becomes many. Encoding so versions can coexist, copying data, splitting data — and why every guarantee above is weaker than it looks.

1

Data outlives code

Old and new versions read each other's data.
Compatibility in both directions.

2

Same data, many machines

Leaders, followers, lag, conflicts, quorums.

3

Different data, many machines

Partition keys, skew, hot spots, rebalancing, routing.

4

Nothing you're told is reliable

Partial failure, unreliable clocks, zombies, fencing.

Print double-sided, short-edge bind, A4 · 100% scale (no “fit to page”).

Answer key starts at Section 9 — keep a sheet of paper over it.

How to use this booklet

The loop, per section

1. Read the plain-version box first. If it doesn't land, the rest won't either.
2. Read the teaching. Say each boundary out loud: “X is *this*, its look-alike Y is *that*.”
3. Cover the page. Write the answer to **Q1** in the blank lines, in your own words, in full sentences. **Then** check the key.
4. Score yourself on completeness. Then do **Q2** and note whether it beat Q1.
5. Only then move on.

THE RULE THAT DOES MOST OF THE WORK

If you find yourself thinking “yes, I know this” — that is *recognition*, and it is not learning. The only evidence that counts is a written answer produced with the page covered.

The spacing schedule

Review at **day 1, day 3, day 7, day 16, day 35**. On a pass, jump to the next interval. On a fail, reset that item to 1 day. Use the grid at the back; one row per item.

What's in here

1 — Learning goal and roadmap	orientation
2 — Data outlives code	teaching · questions · matrix
3 — Same data, many machines	teaching · questions · matrix
4 — Different data, many machines	teaching · questions · matrix
5 — Nothing you're told is reliable	teaching · questions · matrix
6 — Concept sketch + master matrix	consolidation
7 — Symptom → diagnosis → move	working reference
8 — Numbers worth remembering · Glossary	reference
9 — Answer key	keep covered
10 — Spaced-review tracker · final quiz	workbook

ONE WARNING BEFORE YOU START

This booklet teaches the load-bearing spine of a large subject, not all of it. Each part carries the mechanisms you will actually reason with under pressure — the compatibility directions, the three replication families, the sharding schemes and their failure shapes, and the impossibility results underneath all of them. Where a topic is named but not taught, that is deliberate and marked. If you finish able to recite the mechanisms but unable to say *what each one costs*, the booklet has failed.

Learning goal & roadmap

LEARNING GOAL — WHAT YOU SHOULD BE ABLE TO DO WHEN YOU CLOSE THIS

Given a real design or incident (“can we deploy this schema change?”, “why did that user see their own edit vanish?”, “one shard is at 90% CPU and the rest are idle”, “the node came back and corrupted the file”), you can:

1. **Name the mechanism** — which compatibility direction, which replication family, which sharding scheme, which impossibility.
2. **Locate the asymmetry**: which side is old and which is new, which node accepts writes, which key is hot, which observer can't tell slow from dead.
3. **Prescribe the fix** and name the near-miss you rejected.
4. **State the price**. Every one of these costs latency, availability, consistency, operational complexity, or the ability to change your mind later.

The core model in one paragraph

A single machine is a comfortable lie: operations are deterministic, and a fault usually takes the whole thing down rather than leaving you half-working. The moment you have more than one machine, three things stop being true. **Code stops being uniform** — during any rolling upgrade, old and new versions run simultaneously and must read each other's data. **State stops being single** — you either keep copies of the same data (replication) or split different data across machines (sharding), and both introduce divergence. And **knowledge stops being available** — a node cannot tell a slow peer from a dead one, its clock cannot be trusted to order events, and it may itself be paused for a minute without noticing. Parts 1 to 3 build the machinery. Part 4 is the ground truth underneath, and it is the reason every guarantee in the first three parts is hedged.

THE WHOLE THING FOR A SMART 15-YEAR-OLD

- Data you save today gets read by programs written years from now — and by programs written years ago that are still running.
- Once you have more than one computer, you either keep copies of everything everywhere, or you split it up. Both go wrong in their own way.
- Copies drift apart. Splits go lopsided, and one popular item can ruin the whole scheme.
- And you can never tell whether the other machine is broken or just slow. Not with a better timeout — *ever*.

The four parts, and why they're in this order

PART	WHAT IT COVERS	WHY HERE
1 Data outlives code	Backward and forward compatibility; textual versus schema-based binary encodings; how Protocol Buffers and Avro each survive schema change; dataflow through databases, services and message brokers; why remote calls are not local calls.	The problem starts before you distribute anything: two versions of your own code must already share data. Everything later inherits this constraint.

2 Same data, many machines	Single-leader, multi-leader and leaderless replication; synchronous versus asynchronous; failover and split brain; replication lag and its three anomalies; conflict handling; quorums.	The first and most common way to use many machines. It buys availability and read scale, and charges you in consistency.
3 Different data, many machines	Partition keys; key-range versus hash sharding; skew, hot shards and hot keys; rebalancing schemes; request routing; secondary indexes.	The second way, for when the data no longer fits. It answers a different problem from replication, and the two are almost always combined.
4 Nothing you're told is reliable	Partial failure; unbounded network delay; time-of-day versus monotonic clocks; process pauses; the majority rule; leases, zombies and fencing tokens; Byzantine faults; system models.	The floor beneath the other three. Every hedge in parts 1–3 — “eventually”, “probably up to date”, “assumed dead” — is cashed out here.

“The major difference between a thing that might go wrong and a thing that cannot possibly go wrong is that when a thing that cannot possibly go wrong goes wrong, it usually turns out to be impossible to get at or repair.”

Data outlives code

Compatibility in both directions, and the two ways a schema survives being changed.

TENTH-GRADER VERSION

- You save data today. Something reads it in five years — and something written five years ago is still running and reading it too.
- So data has to be readable by code that is *newer* than it, and by code that is *older* than it.
- The second one is the hard direction: the old program was written before the new format existed, and you can't go back and teach it.
- Which is why you can add things to a format safely, but removing or renaming things is where people get hurt.

Start with the recipe box

- A family keeps recipes on index cards in one box. Your grandmother's cards are in the box. So are yours. So will your niece's be.
- You must be able to read your grandmother's cards — she wrote them before you existed. That direction is **easy**, because you can look at an old card and work out what it means.
- Your grandmother must be able to read *your* cards. That direction is **hard**, because she wrote her reading habits decades ago. If you invent a new field — “air fryer temperature” — she has to not choke on it, and crucially, she has to **copy it out unchanged** if she ever rewrites the card, or the information is lost for everyone after her.
- The box is your database. The cards are records. The people are versions of your code, and they are all in the kitchen at the same time.

*“A database generally allows any value to be updated at any time... you may have some values that were written five milliseconds ago and others that were written five years ago... This observation is sometimes summed up as **data outlives code.**”*

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Backward compatibility	Newer code can read data written by older code.	Your new release reads records written last year.	Normally the easy direction: the new code's authors knew the old format existed and could write for it. Confusing the two directions is the single most common error in this area — fix the phrasing by asking <i>whose code is doing the reading.</i>
Forward compatibility	Older code can read data written by newer code.	An instance that hasn't been upgraded yet reads a record written by one that has.	The hard direction, and the one rolling upgrades force on you: the old code was written before the new format existed. It cannot be taught, so the <i>format</i> must be designed so old readers can skip what they don't recognise — and must preserve it if they write the record back.

Rolling upgrade	Deploying a new version to a few nodes at a time, monitoring, and working through the fleet.	Also called a staged rollout.	It's what makes both compatibility directions <i>simultaneously</i> necessary. With client-side applications it's worse: you're at the mercy of the user, who may never install the update.
Field tag Protocol Buffers	A number identifying a field in the encoded data, in place of its name.	<code>string user_name = 1;</code> — the encoded record carries the tag <code>1</code> , never the name.	You can freely change a field's name , since the encoding never refers to names — but you can never change or reuse its tag , because that would make all existing data invalid. Reserve retired tags in the schema so nobody forgets.
Writer's / reader's schema Avro	The schema used to encode, and the schema the reading application expects. Avro resolves the differences by matching fields by name .	The writer's schema must be identical to the one used for encoding; the reader's may be an older or newer version.	vs Protocol Buffers , where a single schema version does both jobs. Avro's consequence: the data itself identifies nothing — no field names, no types — so it can be decoded <i>only</i> with the writer's schema, which must be conveyed somehow.
Schema evolution	Changing a schema over time while keeping old data readable.	Adding a field; retiring a field; widening an integer.	It gives the flexibility of schemaless storage <i>plus</i> better guarantees and tooling. But keep the number of concurrent schema formats to a minimum, or operations get complicated.
Location transparency	The idea behind remote procedure calls: make a network call look like a local function call.	EJB, Java RMI, DCOM, CORBA, SOAP.	Convenient-seeming and fundamentally flawed . A local call is predictable and fails on parameters you control; a network call fails for reasons entirely outside your control, can time out with no result at all so you don't know whether it happened, may need deduplication because a retry can repeat an action that did succeed, and has wildly variable latency.

How each format survives change

CHANGE	PROTOCOL BUFFERS	AVRO
Add a field	Give it a new tag . Old code meets an unrecognised tag and skips it — the datatype annotation tells the parser how many bytes to skip, so it can also <i>preserve</i> the unknown field. That's forward compatibility. New code reading old data fills a default.	Only if the field has a default value . A reader on the new schema fills the default when the old writer omitted it.
Remove a field	Same concerns reversed. Never reuse the tag number — data containing it may still exist and must be ignored by new code.	Only if the field has a default value . Removing a field with no default breaks forward compatibility, because old readers expect it.
Rename a field	Free — the encoding carries tags, not names.	Possible via aliases in the reader's schema, which makes it backward compatible but not forward compatible .
Change a datatype	Sometimes, with a truncation risk: widening a 32-bit integer to 64-bit means new code reads old data fine (missing bits filled with zeros), but old code reading a large new value truncates it into its 32-bit variable.	Possible where Avro can convert the type. Adding a branch to a union is backward but not forward compatible .

Nullability	Absent fields are simply omitted.	Requires an explicit union type — <code>union { null, long, string }</code> — and <code>null</code> can be a default only if it is the first branch . More verbose, deliberately, to be explicit about what can be null.
--------------------	-----------------------------------	--

HOW THE READER GETS THE WRITER'S SCHEMA AT ALL

Avro's encoding contains no field names or type markers, so it's undecodable without the writer's schema — and including the whole schema per record would erase the space saving. Three answers, by context: a **large file of many records** puts the schema once at the start (Avro object container files); a **database of individually written records** stores a version number in each record and keeps a registry of schema versions (Confluent's schema registry for Kafka, LinkedIn's Espresso); **two processes on a connection** negotiate the schema at setup and use it for the connection's lifetime.

Keeping a registry is worth it regardless: it documents the schemas and lets you **check compatibility before anything is deployed**.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Textual formats (JSON, XML, CSV)	Human readability; no schema to distribute.	Size, and no compatibility checking. The binary JSON variants save little — 66 bytes against 81 for the same record — so it's unclear the loss of readability is worth it.	Interchange across organisations, or debugging.
Protocol Buffers	Compact (33 bytes for the same record). Field tags make add/remove safe and renames free. Code generation for statically typed languages.	Tags must be assigned and managed by hand; a reused tag silently corrupts meaning.	Service APIs and long-lived record formats where humans curate the schema.
Avro	Most compact of the three (32 bytes). No tag numbers, so it is friendlier to dynamically generated schemas — you can regenerate a schema from a relational schema on every export and readers still match by name.	The data is meaningless without the writer's schema, so you must build a way to convey it.	Bulk data exports and pipelines where schemas are generated, not curated.
A schema registry	Documentation that cannot drift, plus pre-deployment compatibility checks.	Another service in the path of decoding.	Any environment with more than a couple of writers.
RPC that hides the network	Familiar programming model.	It hides exactly what you must handle: unpredictable failure, timeouts with unknown outcome, the need for idempotence on retry, and variable latency.	Never uncritically. Treat every remote call as a remote call.

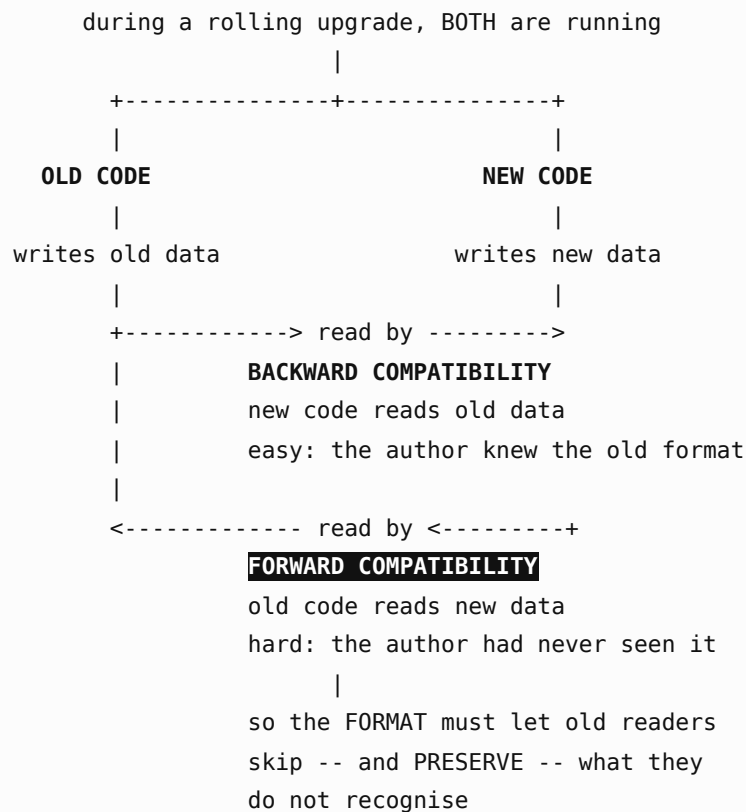
CONCEPT BOUNDARY — WHAT COMPATIBILITY IS *NOT*

It is not a property of your database or your language. A schemaless store does not give it to you: the engine stops caring about structure, but your code still expects it.

It is a relationship between *a process that encodes* and *a process that decodes* — which means it is always directional, and you must say which process is which before the word means anything.

Anchor sketch — the two directions

Legend: **bold** = a named node **reversed** = the node that matters most **Q** = cover the page and answer this



Q Say both definitions out loud without using the words “forward” or “backward” — only “newer code”, “older code”, and which one is reading.

Q Why can't the reversed direction be fixed by patching the old code?

Q What must an old reader do beyond *not crashing* on an unknown field, and what is lost if it doesn't?

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN EXPERT
“The new version can still read the old records.”	“That's backward compatibility — newer code reading data written by older code. The harder one is forward compatibility, which a rolling upgrade forces on us.”
“Old instances choke on the new field.”	“A forward-compatibility break. The encoding needs to let unknown fields be skipped and preserved — that's what the tag plus datatype annotation buys us.”
“We renamed the column.”	“Free in Protocol Buffers, since the encoding carries tags not names. In Avro it needs a reader-schema alias, which is backward compatible but not forward compatible.”
“We can reuse that field number now nobody uses it.”	“Never. Data containing the old tag may still exist and must be ignored — reserve retired tags in the schema.”
“It's schemaless so we don't have to worry.”	“Schemaless is a property of the engine, not the application. Our code still expects structure, and data outlives code.”
“The service call timed out.”	“So we don't know whether it happened. That's the outcome a local function call can't produce, and it's why retries need idempotence.”

PART 1 · QUESTION 1

A team adds a `discount_code` field to an order record and deploys across a fleet in a rolling upgrade. Orders created by upgraded instances carry the field. Support later finds orders where the discount was applied at checkout but the field is missing from the stored record entirely — always orders that were later modified, never orders created and left alone.

(a) Name the compatibility direction that broke and state it without using the words “forward” or “backward”. **(b)** Explain precisely why only *modified* orders lost the field. **(c)** Name the property the encoding must have to prevent this, and say why patching the old code is not the fix.

PART 1 · QUESTION 2

A team dumps a relational database to files nightly for analytics. The relational schema changes every few weeks — columns added, occasionally renamed. They currently hand-maintain a Protocol Buffers schema for the dump and an engineer updates the tag mapping each time a column changes. It is error-prone and twice someone has reused a tag.

(a) Name the format better suited to this job and the single structural property that makes it so. **(b)** Explain how a reader copes when the export’s field set changes, and what it matches on. **(c)** Name the new obligation this format creates that Protocol Buffers does not, and give the mechanism appropriate to *this* use case.

2×2 — who is reading whose data

COLUMNS: WHICH VERSION WROTE THE DATA? →

	Written by OLD code	Written by NEW code
Read by NEW code	Backward compatibility The new release reads last year's records. <i>Move:</i> normally free — the new code's author knew the old format. In Protocol Buffers, missing fields get defaults; in Avro, the reader's schema supplies the default. Only fails if you removed something the new code needs.	Tomorrow. Fine. Both sides current. <i>Move:</i> nothing. This is the state you reach after the upgrade completes and after old data has been rewritten or aged out — which for a database may be never.
Read by OLD code	Yesterday. Fine. The baseline the other cells are measured against. <i>Move:</i> nothing.	Forward compatibility A not-yet-upgraded instance reads a record written by an upgraded one — guaranteed to happen during every rolling upgrade. <i>Move:</i> the <i>format</i> must carry the fix, because the reading code is already deployed and cannot be taught. Old readers must skip unknown fields and preserve them on rewrite . Protocol Buffers: new tags, never reused. Avro: add or remove only fields that have a default.

Read it as: the diagonal is free and the two off-diagonal cells are the whole subject. The top-left is usually solved by whoever writes the new code; the bottom-right cannot be, which is why it is the focus and why encodings are designed around it.

CARRY-OUT RULE FROM THIS PART

Name the reader before you say “compatible”. Compatibility is a relationship between an encoder and a decoder, so it has a direction — and the direction you cannot fix by deploying is the one that determines what your format must do.

Same data, many machines

Leaders, followers, lag and its anomalies, conflicts, and quorums.

TENTH-GRADER VERSION

- Keep copies of the data on several machines: closer to users, still working if one dies, and more machines to answer reads.
- Copying data that never changes is trivial. All the difficulty is in copying *changes*.
- Copies fall behind, so a reader can see an older version of the world — including one where their own edit hasn't happened yet.
- And if more than one copy accepts changes, two people can change the same thing at once, and somebody has to decide who wins.

Start with the fridge list

- A shared shopping list lives on the fridge. One list, everyone writes on it, everyone reads it. No disagreement is possible — but you have to be in the kitchen.
- So everyone photographs it. Now you can read it from the shop — but your photo is from this morning, and someone has added milk since. You're reading a **stale replica**.
- Worse: you add bread to *your photo* while your flatmate adds bread to *theirs*. Now there are two lists that disagree, and nothing in the system knows which is right. That is what accepting writes in more than one place costs.
- The three replication families are three answers to “who is allowed to write on the fridge”: exactly one machine, several machines, or no designated one at all.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Single-leader replication	One replica is the leader and takes all writes; it ships a change stream to the followers, which apply the writes in the same order . Clients may read from any replica.	PostgreSQL, MySQL, Oracle Data Guard, MongoDB, Kafka. Consensus algorithms like Raft also elect a single leader.	vs multi-leader : writes go to exactly one node, so ordering is decided in one place — which is what makes strong guarantees possible at all. If the database is sharded, each shard has one leader , and different shards may lead on different nodes.
Synchronous / asynchronous	Whether the leader waits for a follower's confirmation before reporting success.	Semisynchronous in practice: one follower synchronous, the rest asynchronous, promoting another if the synchronous one stalls.	Making <i>all</i> followers synchronous is impracticable — any one node outage halts every write. Fully asynchronous means a write confirmed to the client is not guaranteed durable if the leader dies; the upside is the leader keeps working however far behind the followers fall.
Failover	Promoting a follower when the leader fails: detect, choose, reconfigure.	Detection is almost always a timeout — no response for, say, 30 seconds means assumed dead.	vs catch-up recovery , which is the easy follower-failure case: a follower reconnects and requests the changes it missed. Failover is the hard case, and every step of it can go wrong.

Split brain	Two nodes both believing they are the leader.	If both accept writes with no conflict resolution, data is likely lost or corrupted.	Guarding against it by limiting or shutting down old leaders is called fencing (part 4). Careless mechanisms can shut down <i>both</i> nodes — and by the time split brain is detected, corruption may already have happened.
Replication log	How the leader tells followers what changed. Three implementations.	Statement-based (ship the SQL), WAL shipping (ship the storage engine's write-ahead log), logical / row-based (ship row-level change records — MySQL's binlog).	Statement-based breaks on nondeterminism — <code>NOW()</code> , <code>RAND()</code> , autoincrements, triggers. WAL shipping is tightly coupled to the storage engine , so leader and follower usually can't run different versions, which forbids zero-downtime upgrades. The logical log is decoupled, so it can stay backward compatible — and it's parseable by outside systems, which is change data capture .
Eventual consistency	Stop writing, wait, and the followers catch up.	Normally under a second of lag; near capacity or during network trouble, seconds to minutes.	“Eventually” is deliberately vague — there is no limit on how far a replica can fall behind. And it is not a NoSQL property: an asynchronously replicated relational follower behaves exactly the same way.
Quorum leaderless	With n replicas, a write must be confirmed by w nodes and a read must query r . If $w + r > n$, the read and write sets overlap, so at least one node you read has the latest value.	$n=3$, $w=2$, $r=2$ tolerates one node down. $n=5$, $w=3$, $r=3$ tolerates two. Common choice: n odd, $w = r = (n+1)/2$.	Quorums need not be majorities — it matters only that the read and write sets overlap. And even with $w + r > n$ the guarantee is soft: concurrent reads and writes, failed writes that partially succeeded, restored-from-old-backup nodes, and rebalancing can all break it. Treat w and r as tuning the <i>probability</i> of staleness, not as absolutes.

The three anomalies of replication lag, and what each needs

ANOMALY	WHAT THE USER SEES	THE GUARANTEE, AND HOW TO GET IT
Reading your own writes	You post a comment, the page reloads from a lagging follower, and your comment is gone. To you it looks like the data was lost.	Read-after-write consistency. Read anything the user may have modified <i>from the leader</i> (simple rule: own profile from the leader, others' from a follower). If too much is user-editable, read from the leader for a minute after any write, or track the timestamp of the user's last write and only serve them replicas that have caught up to it. Cross-device makes it harder: the metadata must be centralised, and different devices may route to different regions.
Monotonic reads	You refresh and see a comment; you refresh again, are routed to a laggier replica, and it's gone. Time appears to move backward.	Make each user always read from the <i>same</i> replica — for example, chosen by a hash of the user ID rather than at random.
Consistent prefix reads	You see an answer before the question it answers. A causal ordering is violated across shards.	Ensure causally related writes are written to the same shard — or track causality explicitly.

*A GitHub incident: an out-of-date MySQL follower was promoted to leader. The database used an autoincrementing counter for primary keys, and because the new leader's counter lagged, it **reused primary keys** the old leader had already assigned. Those keys were also used in a Redis store, so the reuse caused inconsistency between the two — and **private data was disclosed to the wrong users**.*

That is what “discarding the old leader's unreplicated writes” means in practice, and it is why the most important thing in a failover is picking an up-to-date follower: with semisynchronous replication, the one the leader waited for; with asynchronous, the one with the highest log sequence number.

When more than one node accepts writes

- **Multi-leader** is worth its complexity in three situations: **geo-distributed** deployments where every region should take local writes and hide the inter-region delay; **offline-capable clients**, where each device is a leader and replication lag may be hours or days; and **real-time collaboration**, where each browser tab holding a shared document is a replica. The libraries doing this are **sync engines**; an app that keeps working offline is **offline-first**, and one designed to keep working even if the vendor shuts down its services is **local-first**.
- The price is stated bluntly by the source: the consistency multi-leader can achieve is **much weaker**. You cannot guarantee that a bank account won't go negative, or that a username is unique, because two leaders can each accept a write that is individually fine and jointly illegal. If you need such constraints you are better off with a single leader — this is a **fundamental limitation of distributed systems**, not a gap in any product.
- **Topologies** — circular, star, all-to-all — decide the paths writes take. Circular and star break if one node fails, since they route through it; all-to-all is more fault tolerant but lets messages **overtake each other**, so a replica can receive an update to a row before the insert that created it. Loops are prevented by tagging each write with the identifiers of the nodes it has passed through.
- **Leaderless** (Dynamo-style: Riak, Cassandra, ScyllaDB) abandons the leader entirely — clients write to several replicas at once, and there is **no such thing as failover**. Missed writes are repaired by **read repair** (a reading client notices a stale replica and writes the newer value back), **hinted handoff** (another replica holds writes on behalf of an absent one and delivers them later), and **anti-entropy** (a background process that copies missing data, in no particular order and possibly much later).

- The structural advantage: a leaderless system **does not distinguish the normal case from the failure case**. There is no failover decision to get wrong, so one slow or absent replica barely affects response times — the client just uses the faster responses, a technique called **request hedging**. This is especially valuable against **gray failures**, where a node is not down but is degraded and unusually slow.

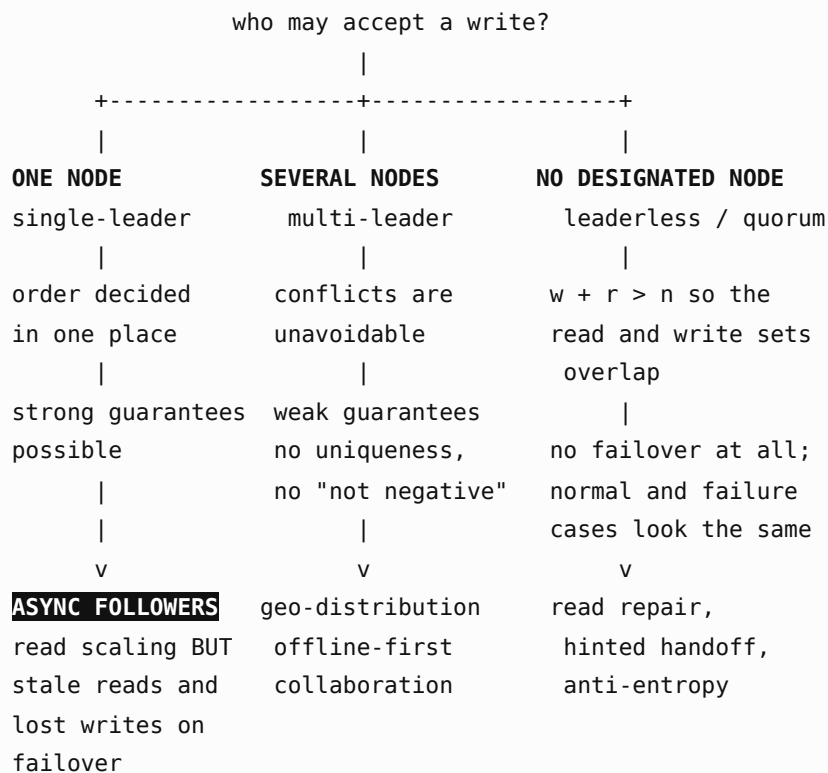
CONCEPT BOUNDARY — REPLICATION IS NOT BACKUP

It is not a way to recover from mistakes. Replicas reflect writes from one node onto others as fast as they can, so if you accidentally delete data, **the deletion is faithfully propagated too**.

It is a way to survive machines failing. Backups store *old snapshots* so you can go back in time — a different purpose, and complementary: backups are often part of setting up a new replica, and archiving replication logs can be part of a backup process.

Anchor sketch — three answers to “who may write?”

Legend: bold = a named node **reversed = focus node** Q = self-test cue



Q Why is the reversed node the one you are most likely to be running without having chosen it?

Q Name the two constraints the middle branch structurally cannot enforce, and say why.

Q Give the quorum inequality, and one reason it can hold and still return a stale value.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Synchronous follower	A guaranteed up-to-date copy on at least two nodes.	If the synchronous follower stalls, all writes block until it returns.	Durability matters more than write availability — and then only semisynchronously.
Fully asynchronous	The leader keeps accepting writes however far behind followers fall; read scaling works.	Confirmed writes can be lost on failover; readers see stale data.	Many followers, or geographically distributed ones. Which is most deployments.
Automatic failover	Recovery without waiting for a human.	Timeout choice is a trap: too long and recovery drags, too short and you get unnecessary failovers — and an unnecessary failover during a load spike makes things worse . Some teams deliberately prefer manual failover.	The failure rate justifies the risk, and fencing is in place.
Multi-leader	Local writes in every region; offline and collaborative apps become possible.	Conflicts, weak consistency, and it's often a retrofitted feature — autoincrementing keys, triggers and integrity constraints interact badly. Widely considered dangerous territory.	Geo-distribution, offline-first, or collaboration — where there is no alternative.
Leaderless with quorums	No failover; resilience to slow and gray-failing nodes; tunable staleness.	Weaker guarantees than they appear; harder to monitor, because with no fixed write order there is no simple lag metric.	You can tolerate eventual consistency and value availability under partial failure.
Bigger quorums	Lower staleness probability.	More responses to wait for, so a higher chance of hitting a slow replica. In practice quorums are seldom more than four of seven or five of nine .	Rarely worth pushing past those sizes.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN EXPERT
“The copy was out of date.”	“Replication lag on an asynchronous follower — eventual consistency, with no upper bound on how far behind it can fall.”
“The user couldn't see their own comment.”	“A read-after-write violation. Route reads of anything the user may have modified to the leader, or track their last-write timestamp and only serve replicas that have caught up.”
“It showed up, then disappeared on refresh.”	“A monotonic reads violation — the second read hit a laggier replica. Pin each user to one replica, by hash of user ID.”
“We lost writes when it failed over.”	“Asynchronous replication means unreplicated writes are discarded on promotion — writes we'd confirmed to clients turn out not to have been durable.”
“Both nodes thought they were in charge.”	“Split brain. It needs fencing, and shutdown-based mechanisms can take out both nodes or act too late.”
“We'll let both regions accept writes.”	“Multi-leader. Accept that we can no longer guarantee uniqueness or non-negative balances — that's a fundamental limitation, not a product gap.”
“We write to two of three and read from two of three.”	“A quorum with $w + r > n$, so the read and write sets overlap. It tunes staleness probability rather than guaranteeing freshness.”

“We have replicas, so we're covered if someone deletes something.”

“No — the deletion replicates too. Backups serve a different purpose: going back in time.”

PART 2 · QUESTION 1

A social app runs one leader and six asynchronous followers, load-balancing reads randomly. Users report three things: their own new posts sometimes vanish on the next page load; a friend's comment appears, then disappears on refresh, then reappears; and after a failover last month, roughly forty seconds of posts were gone permanently.

(a) Name each of the three symptoms precisely. **(b)** Give the specific fix for each of the first two, and say why the fix for the second does not solve the first. **(c)** Explain the third and name the single most important decision during failover that limits it.

PART 2 · QUESTION 2

A team wants users in three regions to get fast writes, so they propose a leader in each region replicating asynchronously to the others. The product requires that usernames be globally unique and that a prepaid balance can never go negative. An architect suggests “we'll just add a uniqueness constraint and a check constraint”.

(a) Name the architecture and explain, mechanically, why both constraints are unenforceable under it. **(b)** Say whether this is a limitation of the database product, and justify the answer. **(c)** Give the two structural options, name one topology hazard they should know about regardless, and say which option you'd take.

2×2 — the replication design space

COLUMNS: DOES A WRITE WAIT FOR OTHER REPLICAS BEFORE BEING CONFIRMED? →

	Waits synchronous	Doesn't wait asynchronous
ONE node accepts writes	Semisynchronous leader One synchronous follower, the rest asynchronous; promote another if it stalls. <i>Move:</i> the durable choice — an up-to-date copy on at least two nodes. Price: if the synchronous follower stops responding, writes block until it returns, so never make them all synchronous.	Asynchronous leader The overwhelmingly common deployment, and often not a deliberate choice. <i>Move:</i> accept that you have eventual consistency and handle its three anomalies explicitly, and accept that confirmed writes can be discarded on failover. Choose the most up-to-date follower when promoting.
SEVERAL nodes accept writes	Degenerate Synchronous multi-leader: if a write to A must reach B before succeeding, an interruption between them blocks writes anyway. <i>Move:</i> recognise this collapses into single-leader — you have paid multi-leader's complexity for single-leader's behaviour. Pick one deliberately.	Multi-leader and leaderless Geo-distribution, offline-first, collaboration; or Dynamo-style quorums with no leader at all. <i>Move:</i> conflict resolution is now <i>your</i> problem, and uniqueness and threshold constraints are off the table. In exchange you get local writes, no failover, and resilience to gray failure.

Read it as: going down buys availability and costs consistency; going right buys latency and costs durability. The bottom-left cell is the only one nobody should deliberately occupy — everywhere else is a real trade.

CARRY-OUT RULE FROM THIS PART

Count the places a write can be accepted. One place means ordering is decided for you and strong guarantees are available. More than one means conflicts exist whether or not you have designed for them — and constraints that span records, like uniqueness, quietly stop being enforceable.

Different data, many machines

Partition keys, skew and hot spots, the sharding schemes, and what rebalancing costs.

TENTH-GRADER VERSION

- When the data no longer fits on one machine, split it up and put each piece somewhere different.
- Splitting is easy. Splitting so that every machine gets a *fair share of the work* is the hard part.
- One extremely popular item can land all the traffic on one machine while the others sit idle — and no clever splitting scheme fixes that by itself.
- And you have to choose how to split before you know how people will use it. Changing your mind later is expensive.

Start with the encyclopaedia

- A printed encyclopaedia is split across twelve volumes by the first letters of the entry. To find *Tapir* you go straight to the right volume — no searching.
- Notice the volumes aren't evenly lettered: volume 1 covers A–B, volume 12 covers T through Z. The **boundaries adapt to the data**, because words aren't evenly distributed across the alphabet.
- Now imagine the entries were filed by *date added* instead. Every new entry goes into the last volume, so one volume is constantly in use and eleven sit on the shelf. That's a **hot shard**, and it's caused by the key, not the scheme.
- The fix — hash the key first — makes every volume equally busy but destroys the thing that made the encyclopaedia useful: you can no longer fetch a range in one place.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Sharding partitioning	Splitting data so each record lives on one shard, to spread data volume and write throughput across machines.	Usually combined with replication, so copies of each shard live on several nodes.	vs replication : replication copies the <i>same</i> data; sharding distributes <i>different</i> data. If read throughput is your only problem you may not need sharding at all — read scaling from part 2 may be enough. Also unrelated to a network partition , which is a network fault.
Partition key	The value that decides which shard a record goes to. All records with the same partition key land on the same shard.	The key in a key-value store; a column — not necessarily the primary key — in a relational model.	The choice matters permanently: access is fast if you know the shard, and an inefficient search across all shards if you don't. The sharding scheme is also difficult to change afterwards.
Skew · hot shard · hot key	Unfair distribution; a shard with disproportionate load; a single key with disproportionate load.	A celebrity's post drawing a storm of reads and writes to one key.	Skew is the condition, a hot shard is where it lands, a hot key is a cause you cannot hash away — consistent hashing distributes keys uniformly, which is not the same as distributing load uniformly .

Key-range sharding	Each shard owns a contiguous range of partition keys; keys are stored sorted within a shard.	Bigtable, HBase, CockroachDB, FoundationDB, MongoDB's range option. Manual variant in Vitess.	Its advantage is efficient range scans — fetch a month of sensor readings in one query. Its matching weakness is a hot shard whenever writes cluster on nearby keys, which a timestamp key guarantees. The standard fix is to prefix the timestamp with something spreading, like a sensor ID — at the cost of needing a separate range query per sensor.
Hash sharding	Hash the partition key first, then assign by hash.	MongoDB uses MD5; Cassandra and ScyllaDB use Murmur3.	The hash need not be cryptographically strong, but it must be stable across processes — Java's <code>Object.hashCode()</code> and Ruby's <code>Object#hash</code> can differ between processes, which makes them unusable here. The cost of hashing is that range queries over the partition key are no longer efficient .
Rebalancing	Moving data between nodes so load stays even as nodes are added or removed.	Fixed shard count, hash ranges, or splitting oversized shards.	Never use hash mod N over the node count: change N and most keys must move. The whole design problem is moving <i>as little data as possible</i> .
Consistent hashing	A hash function that keeps shards roughly equal <i>and</i> moves as few keys as possible when the shard count changes.	Cassandra and ScyllaDB's scheme, plus rendezvous hashing and jump consistent hashing.	The word “ consistent ” here has nothing to do with replica consistency or ACID consistency — it describes a key's tendency to stay in the same shard. This collision of vocabulary is a reliable source of confusion.

The three ways to assign hashes to shards

SCHEME	HOW IT WORKS	WHAT IT COSTS
Hash mod N	<code>hash(key) % number_of_nodes</code> .	Don't. Add a fourth node to three and most keys move — hugely inefficient rebalancing for no benefit.
Fixed number of shards	Create far more shards than nodes and assign several to each — 1,000 shards over 10 nodes is 100 each. Adding a node reassigns whole shards; the key-to-shard mapping never changes.	Only entire shards move, which is cheap. But you must guess the count up front: you can never have more nodes than shards, and getting it wrong means an expensive resharding . Shard size grows with the dataset, so a fixed count suits a predictable total size. Pick a number divisible by many factors.
Hash-range sharding	Give each shard a contiguous <i>range of hash values</i> . Shards split when too big or too hot, so the count adapts.	Splitting is still expensive, but happens on demand. Used by YugabyteDB and DynamoDB; Cassandra and ScyllaDB use a variant with random boundaries and several ranges per node — 16 per node by default in Cassandra, 256 in ScyllaDB — so imbalances even out.

HOT KEYS: THE PROBLEM HASHING CANNOT SOLVE

Uniformly distributed keys do not mean uniformly distributed *load*. If one key is genuinely hot, a range-based scheme can at least isolate it — put it in a shard by itself, perhaps on a dedicated machine.

At the application level, the standard trick is to **add a random number to the key**: two random digits split writes across 100 keys. But be honest about what it buys. Reads must now query all 100 keys and combine the results, so **only the write load is split — the read volume is unchanged**. It needs bookkeeping to track which keys are split, and load shifts over time: a viral post is hot for days and then isn't, and some keys are hot for reads while others are hot for writes.

Rebalancing: automatic or manual?

- Fully automatic rebalancing means less operational work and enables autoscaling — cloud databases advertise adding and removing shards within minutes.
- But rebalancing is **expensive**: it reroutes requests and moves large amounts of data. Done carelessly it overloads the network or the nodes. And the system must keep serving writes throughout — near maximum write throughput, the splitting process **may not keep up with incoming writes at all**.
- The genuinely dangerous interaction is with automatic failure detection. A node is overloaded and slow; the others conclude it is dead and rebalance load away from it; that **adds** load to everyone else and the network; more nodes are then falsely suspected. This is a **cascading failure** caused entirely by the automation.
- Hence a human in the loop is often worth the slowness — and manual rebalancing lets you pre-emptively prepare for a known event, like a holiday sale or a ticket release.

Secondary indexes and multitenancy — two things sharding makes harder

- Sharding works well for key-value data. It is harder for relational data, where you may want to search by a **secondary index** or join records that live on different shards. A write may also need to update related records in several shards, which requires a **distributed transaction** — available in some databases, but usually much slower than single-node transactions and capable of becoming a system-wide bottleneck.

- **Multitenancy** is a common non-scalability reason to shard: give each tenant a shard, or group small tenants together. It buys **resource isolation** (one tenant's expensive query doesn't hurt others), **permission isolation** (a bug in access control is less likely to cross tenants when the data is physically separate), **per-tenant backup and restore**, straightforward **GDPR-style export and deletion**, **data residency**, and **gradual schema rollout** one tenant at a time. Extended to services as well as storage, it becomes a **cell-based architecture**, where a fault stays inside its cell.
- The catches: it assumes each tenant fits on one node (if not, you must shard within the tenant too), many small tenants make per-tenant shards wasteful, and any feature that joins data *across* tenants becomes hard.

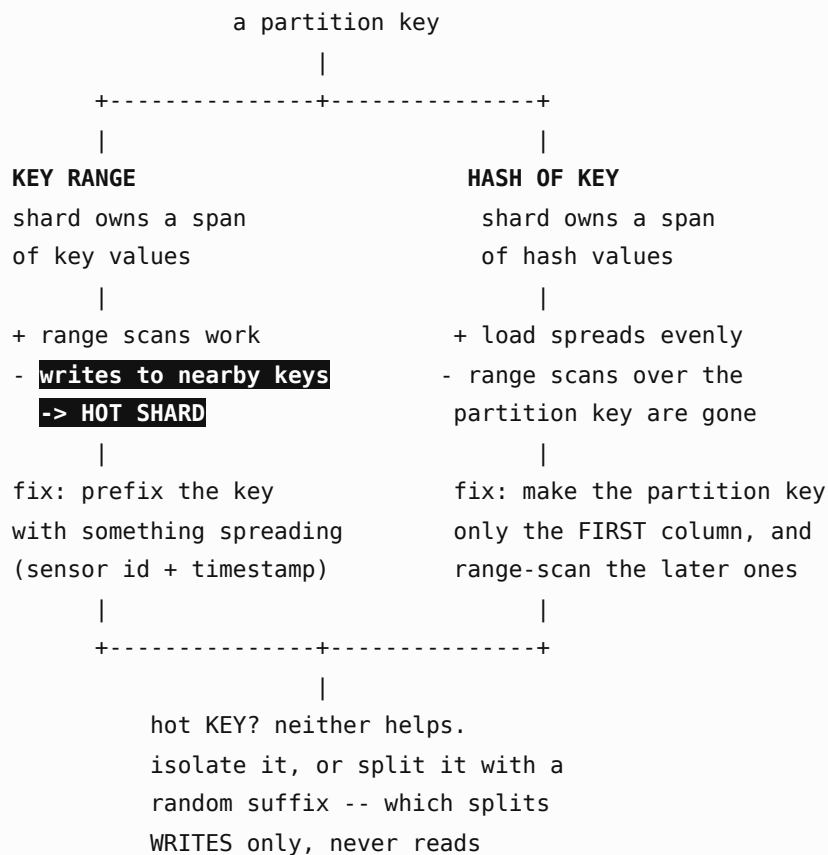
CONCEPT BOUNDARY — SHARDING IS NOT THE DEFAULT ANSWER TO “WE NEED TO SCALE”

It is not a lightweight tuning step. It adds real complexity: a partition key you must choose before you understand the workload and can barely change later, cross-shard queries that are slow, and cross-shard writes that need distributed transactions.

It is a heavyweight solution mostly relevant at large scale. Replication is useful at every scale; sharding earns its place only when data volume or write throughput genuinely exceeds one machine — and a single machine can do a great deal.

Anchor sketch — choosing a scheme

Legend: **bold** = a named node **reversed** = focus node Q = self-test cue



Q Why does hashing solve a hot *shard* but not a hot *key*?

Q The random-suffix trick splits writes across 100 keys. What happens to the read load, and what bookkeeping does it force?

Q Name the one rebalancing scheme you should never use, and the specific reason.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN EXPERT
“One server is doing all the work.”	“A hot shard — the workload is skewed. Is it the scheme putting nearby keys together, or a genuinely hot key?”
“We split by timestamp.”	“Then all current writes land on one shard. Prefix the key with something spreading and accept a separate range query per prefix.”
“We hash the key across the nodes.”	“Not mod N over the node count — change N and most keys move. Use a fixed shard count, or hash ranges that split on demand.”
“Consistent hashing keeps the data consistent.”	“Different sense of the word — it means keys tend to stay in the same shard when the shard count changes. Nothing to do with replica or ACID consistency.”
“One customer is huge.”	“Then per-tenant sharding breaks its own assumption and you need to shard within that tenant.”
“The cluster rebalanced itself and everything got worse.”	“Automatic rebalancing interacting with automatic failure detection — an overloaded node was declared dead, and moving its load made the rest worse. A cascading failure caused by the automation.”
“That query has to check every shard.”	“It doesn't use the partition key, so it's a scatter-gather. That's the cost of the key choice — and secondary indexes are where sharding stops being cheap.”

PART 3 · QUESTION 1

A fleet-tracking system stores position pings keyed by timestamp, sharded by key range across twelve nodes. The main query is “all pings in the last hour”. One node runs at 95% CPU while the other eleven idle. An engineer proposes switching to hash sharding on the timestamp.

(a) Explain the current hot shard mechanically, and say whether the scheme or the key is at fault. **(b)** State exactly what the proposed fix would break, and why that matters given their main query. **(c)** Give the better fix, and name the query cost it imposes.

PART 3 · QUESTION 2

A marketplace shards orders by `buyer_id` using a fixed 256-shard scheme over 16 nodes. A single institutional buyer now generates 40% of all writes. Someone suggests raising the shard count to 4,096 to spread the load. Separately, the cluster recently “healed itself” during a traffic spike and the outage got worse for twenty minutes.

(a) Explain why raising the shard count will not help, using the right term for the problem. **(b)** Give the two available responses, and state precisely what the application-level one does and does not reduce. **(c)** Explain the self-healing incident, and say what you would change about the rebalancing policy.

2×2 — picking a sharding scheme

COLUMNS: DO WRITES CLUSTER ON NEARBY PARTITION KEYS? →

	Writes spread out	Writes cluster timestamps, sequential IDs
You NEED range scans on the partition key	Key range, comfortably Sorted within each shard, so scans are cheap and boundaries adapt to the data. <i>Move:</i> use key-range sharding and let the database split shards as they grow — HBase's default trigger is a shard reaching 10 GB.	The real tension You want ordered access <i>and</i> your writes all land at one end of the order. Range scans demand the very layout that creates the hot shard. <i>Move:</i> don't hash the whole key — that would destroy the scans you need. Make the partition key a compound : something spreading first, the sequential value second. You keep range scans <i>within</i> each prefix and pay by issuing one query per prefix.
You DON'T need range scans	Hash, easiest case Nothing pulls in either direction. <i>Move:</i> hash sharding. Use a hash that's stable across processes — not Java's <code>Object.hashCode()</code> .	Hash, and it earns its keep Hashing is precisely what turns clustered keys into an even spread. <i>Move:</i> hash sharding, ideally hash- <i>range</i> so shards can split on demand rather than fixing the count up front.

Read it as: the top-right is the only genuinely hard cell, and it is hard because the two requirements are in direct opposition. Note what no cell contains: a **hot key**. That is a workload property, not a scheme property, and it must be handled separately by isolating or splitting the key.

CARRY-OUT RULE FROM THIS PART

The partition key is a bet on the workload, placed before you have seen it. Choose it for how the data will be *read* and *written*, not for how it is naturally identified — and remember that no scheme spreads a single hot key, because the unevenness is in the traffic, not the layout.

Nothing you're told is reliable

Partial failure, unbounded delay, untrustworthy clocks, zombies — and the fence that saves you.

TENTH-GRADER VERSION

- One computer either works or crashes. Many computers can be half-broken, and you can't tell which half.
- You send a message and get nothing back. Was it lost? Is the machine dead? Did the reply get lost? **You cannot tell.**
- Not with a better timeout, not with a faster network — ever. All you can do is guess, and design so a wrong guess doesn't destroy anything.
- You can't even trust your own clock, or trust that your own program hasn't been frozen for a minute without noticing.

Start with the letter

- You post a letter asking a question and hear nothing for a fortnight. What happened? Your letter was lost; or it arrived and they're away; or they've died; or they replied and *their* letter was lost; or it's all just slow.
- From your side, every one of these looks **identical**: no reply. You cannot distinguish them, and no amount of waiting turns a guess into knowledge — waiting longer only trades one error for another.
- So you set a rule: no reply in three weeks, assume they're not coming. That rule is a **timeout**, and it is a *decision*, not a discovery.
- Now the dangerous part. Suppose you assume they're gone and give their job to someone else — and then the original person turns up, having been ill, with no idea they were replaced. Two people now believe they hold the same job. That is the entire safety problem of distributed systems.

“In my limited experience I've dealt with long-lived network partitions in a single data center, PDU failures, switch failures, accidental power cycles of whole racks, whole-DC backbone failures, and a hypoglycemic driver smashing his Ford pickup truck into a DC's HVAC system. And I'm not even an ops guy.”

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Partial failure	Some parts of the system are broken unpredictably while others work fine.	An operation involving several nodes sometimes works and sometimes doesn't — and you may not even know whether it succeeded.	vs a single-computer fault , which is deliberately designed to be all-or-nothing; computers prefer to crash outright rather than return a wrong result. Partial failure's defining property is nondeterminism , and that is what makes distributed systems hard.

Unbounded delay	An asynchronous packet network gives no guarantee about when — or whether — a message arrives.	Round-trip times of several minutes have been observed at high percentiles across cloud regions; even within one datacentre a topology reconfiguration can delay packets by more than a minute.	vs a hypothetical synchronous network with a maximum delay d and a maximum handling time r , where $2d + r$ would be a correct timeout. Real systems have neither guarantee, which is exactly why no timeout is correct.
Time-of-day clock	Calendar time, synchronised by NTP, comparable across machines.	<code>clock_gettime(CLOCK_REALTIME)</code> , <code>System.currentTimeMillis</code> .	vs a monotonic clock : time-of-day can be forcibly reset and jump backward , which makes it unsuitable for measuring elapsed time. Its absolute value is meaningful; a monotonic clock's is not.
Monotonic clock	Always moves forward; good for durations.	<code>CLOCK_MONOTONIC</code> , <code>System.nanoTime</code> . Resolution usually microseconds or better.	Its absolute value is meaningless — perhaps nanoseconds since boot — so comparing monotonic values across two computers is nonsense . NTP may <i>slew</i> its rate by up to 0.05% but cannot make it jump.
Process pause	A running thread stops for an arbitrary time without knowing it did.	A garbage collection pause; a VM paused for tens of milliseconds while another VM uses the core.	The dangerous consequence: on resuming, the code has no idea time has passed and continues as though nothing happened — including acting on a lease that has since expired.
Lease · zombie	A lock that times out and can be reassigned. A zombie is a former holder that hasn't found out it lost the lease.	Used to ensure only one leader per shard, or one writer per file.	You cannot rule zombies out — a pause or a delayed message will always be possible — so the goal is not to prevent them but to ensure they can do no damage.
Fencing token	A number the lock service returns with every grant, increasing each time. Every write must carry it, and the storage service rejects any token lower than one it has already accepted.	Client 1 holds token 33 and pauses; client 2 acquires token 34 and writes; client 1 wakes and writes with 33, and the storage service rejects it .	vs shutting the zombie down (“STONITH”), which is not effective: it can't stop an already-delayed message, and by the time you detect the zombie the damage may be done. Fencing is permanent, unlike optimistic concurrency control, which can be retried. Called <i>sequencers</i> in Chubby, <i>epoch numbers</i> in Kafka.

Byzantine fault	A node that lies — sends arbitrary, faulty or contradictory responses.	Casting two contradictory votes in one election.	vs the assumption made almost everywhere else, that nodes are unreliable but honest : slow, absent, or out of date, but truthful when they do respond. Byzantine tolerance matters in aerospace and among mutually untrusting parties; in a normal datacentre it is impractically expensive and unnecessary.
-----------------	---	--	---

Why you cannot tell slow from dead

- Send a request and get no response. The possibilities are: the request was lost; it is queued and will be delivered later; the node failed; the node is paused but will resume; the node processed it and the *response* was lost; or the response is merely delayed. **These are indistinguishable.** The only information you have is that you haven't received a reply.
- TCP doesn't rescue you. It retransmits and reorders, but it can't tell whether the outbound packet or the acknowledgment was lost, and it can't plug a cable back in. Worse: if a connection closes with an error you have **no way of knowing how much data the remote node actually processed**, and an acknowledgment only proves the remote *kernel* received it — the application may have crashed before handling it. **If you need to be sure, you need a positive response from the application itself.**
- So you use a timeout — and a timeout is a bet with a cost on both sides. Too long, and users wait through an outage. Too short, and you declare live nodes dead: an action may be performed **twice**, and transferring a slow node's load onto its peers can **cascade** until everything stops.
- Network faults are not rare enough to ignore. One study of a medium-sized datacentre found **about 12 network faults per month**, half disconnecting a single machine and half a whole rack. Redundant networking gear helps less than you'd hope, because it doesn't guard against **human error**, a major cause of outages.

CLOCKS ARE WORSE THAN THEY LOOK, AND FAIL SILENTLY

- **Drift.** Google assumes up to **200 ppm** for its servers — about **6 ms** of drift for a clock resynchronised every 30 seconds, or **17 seconds** for one resynchronised daily.
- **Sync quality.** A minimum error of about **35 ms** is achievable over the internet, with occasional spikes to around a second. Some public NTP servers are simply wrong, off by hours.
- **Jumps.** A clock too far from its NTP server may be forcibly reset, so applications watching it see time **go backward**. Leap seconds have crashed many large systems.
- **Silence is the real danger.** A defective CPU or misconfigured network breaks loudly. A defective quartz clock or a misconfigured NTP client lets everything *seem* to work while drifting further from reality — so the result is **silent, subtle data loss rather than a dramatic crash**. Monitor clock offsets and evict any node that drifts too far.
- High accuracy is buyable if you must: MiFID II obliges high-frequency trading funds to stay within **100 microseconds** of UTC, using GPS receivers, atomic clocks and PTP.

Ordering by timestamp: the trap this sets

- Client A writes `x = 1` on node 1; it replicates to node 3; client B increments to `x = 2` on node 3; both replicate to node 2. Clock skew between node 1 and node 3 is under 3 ms — better than you can usually expect.
- Yet `x = 1` carries timestamp **42.004** and `x = 2` carries **42.003**. B's write is *causally later* but has the *earlier* timestamp. Under **last write wins**, node 2 keeps `x = 1` and **the increment is silently lost**.

- Three consequences worth memorising: writes can **mysteriously disappear**, because a node with a lagging clock cannot overwrite a faster node's values until the skew has elapsed; LWW **cannot distinguish** genuinely concurrent writes from sequential ones, which is what version vectors are for; and two nodes can generate **identical timestamps** at millisecond resolution, needing a tiebreaker that itself can violate causality.

What you can rely on instead

- **The majority.** A node cannot trust its own judgment — it may be semi-disconnected, receiving everything while all its outgoing messages are dropped, protesting “I'm not dead!” to nobody. So decisions, *including the decision that a node is dead*, are made by quorum. A majority is safe because **there cannot be two conflicting majorities at once**, and it tolerates a minority failing: one of three, two of five.
- **Fencing tokens.** Since zombies cannot be prevented, make their writes harmless. A monotonically increasing token on every grant, carried on every write, and rejected by the storage service if lower than one already seen. A client that has just acquired a lease should write immediately, which fences off any zombie. If your storage supports a conditional write — S3's conditional writes, Azure's conditional headers, Google Cloud Storage's request preconditions — that is enough on its own for a single service.
- **An explicit system model.** State your assumptions and design to them. On timing: **synchronous** (bounded delay, pauses and clock error — unrealistic), **partially synchronous** (well behaved *most* of the time, with occasional arbitrary violations), **asynchronous** (no timing assumptions at all, not even a clock — very restrictive). On nodes: **crash-stop**, **crash-recovery** (with stable storage surviving, in-memory state lost), **degraded / gray failure** (a limping node still passing health checks while doing no real work), and **Byzantine**. For real systems, **partially synchronous with crash-recovery faults** is generally the most useful.

CONCEPT BOUNDARY — A TIMEOUT DOES NOT DETECT ANYTHING

It is not a measurement. No timeout can distinguish a dead node from a slow one, because the network has no upper bound on delay and the node has no upper bound on its own pauses.

It is a decision to stop waiting and act on a guess — chosen experimentally by measuring the distribution of round-trip times, and traded off between declaring live nodes dead and waiting too long on dead ones. Design so that **being wrong is survivable**, which is what fencing tokens and quorums are for.

Anchor sketch — what you know versus what is true

Legend: **bold** = a named node **reversed** = focus node Q = self-test cue

you send a request. no reply arrives.

|

possible worlds, all identical to you:

request lost . queued . node dead .

node paused . reply lost . reply delayed

|

v

A TIMEOUT -- a decision, not a discovery

|

+-----+-----+

|

too long

users wait through

an outage

|

too short

live nodes declared dead

|

+-----+-----+

|

action done

TWICE

|

load moves to

peers -> cascade

|

v

THE ZOMBIE

still believes it holds the lease

|

cannot be prevented -- pauses and

delayed messages always possible

|

v

FENCING TOKEN

monotonically increasing, carried on

every write, lower ones rejected

Q Give three of the six indistinguishable worlds, and say what a longer timeout would change about them.

Q Why is shutting a zombie down not a solution, even when it works?

Q A quorum declares a healthy node dead. What must that node do, and why is a majority safe?

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN EXPERT
“The server didn't respond, so it's down.”	“We can't conclude that. No reply is consistent with a lost request, a queued request, a pause, a lost reply or a delayed reply — they're indistinguishable in an asynchronous network.”
“We'll tune the timeout properly.”	“There's no correct value — delays are unbounded. Measure the round-trip distribution, pick experimentally, and design so a wrong guess is survivable.”
“We timestamp writes to see which is newer.”	“That's last-write-wins on time-of-day clocks. A causally later write can carry an earlier timestamp and be silently dropped — you need version vectors to track causality.”
“We measure how long it took with the system clock.”	“Use a monotonic clock for durations. Time-of-day can be reset and jump backward.”
“The node came back and overwrote everything.”	“A zombie acting on an expired lease. Fencing tokens make its writes rejectable rather than trying to prevent it existing.”
“It's still responding to health checks so it's fine.”	“That's a gray failure — a limping node passing health checks while too slow to do real work. Harder to handle than a clean crash.”
“The node knows it's still the leader.”	“A node can't trust its own judgment. If a quorum declares it dead it must step down — and there can't be two conflicting majorities at once.”
“What if a node sends bad data on purpose?”	“That's a Byzantine fault. We assume nodes are unreliable but honest; Byzantine tolerance is for aerospace and untrusting parties, and it's impractically expensive here.”

PART 4 · QUESTION 1

A job scheduler uses a lock service so that exactly one worker processes each input file. A worker acquires the lock, begins writing output, suffers a two-minute garbage collection pause, and resumes. Meanwhile the lease expired, a second worker acquired it and wrote its own output. The file is now corrupt. The team proposes reducing GC pauses and doubling the lease duration.

(a) Name what the first worker became and explain why the proposed fix cannot eliminate the problem. **(b)** Name the mechanism that does solve it, describe precisely how it rejects the stale write, and say what a client should do immediately on acquiring a lease. **(c)** Name the popular alternative approach and give two reasons it is ineffective.

PART 4 · QUESTION 2

A multi-region key-value store resolves conflicts by last-write-wins on the client's time-of-day timestamp. An engineer reports that a small number of updates “never take effect” — the write returns success, and a later read shows the old value. It correlates loosely with one region. Clocks are NTP-synchronised and skew is measured at under 5 ms.

(a) Explain the mechanism, and why sub-5 ms skew does not make it safe. (b) Name the second, distinct defect of last-write-wins that this system also has, and the mechanism that addresses it. (c) Say why this class of fault is especially dangerous operationally, compared with a node that crashes.

2×2 — what you believe against what is true

COLUMNS: IS THE NODE ACTUALLY WORKING? →

	It is working	It is genuinely dead
You believe it is ALIVE	Correct, and lucky The normal case. Note it's a belief, not knowledge — you infer it from recent replies. <i>Move:</i> nothing, but don't mistake this for certainty. A gray failure lives here too: still answering health checks, too slow to do real work.	You are waiting on a corpse Requests hang, and no failover starts because nobody has decided it is dead. <i>Move:</i> this is what timeouts are <i>for</i>. Too long a timeout parks you here, so users experience the outage in full.
You believe it is DEAD	The zombie A pause, a semi-disconnection, or an over-eager timeout. The node keeps acting on a lease it no longer holds — and does not know time has passed. <i>Move:</i> you cannot prevent this, so make it harmless. Fencing tokens on every write; decisions by quorum, which the node must obey even feeling perfectly healthy. Beware: an action may now happen twice, and its load moving to peers can cascade.	Correct, and handled Detection worked, failover proceeds. <i>Move:</i> promote the most up-to-date replica, and make sure the old node is fenced before it can return.

Read it as: the diagonal is what you hope for and cannot verify. The two off-diagonal cells are the two ways a timeout is wrong, and they trade directly against each other — every millisecond you shorten it moves risk from the top-right to the bottom-left. The bottom-left is worse, because it corrupts rather than delays.

CARRY-OUT RULE FROM THIS PART

Design for being wrong, not for being right. You will misjudge which nodes are alive, your clocks will disagree, and your own process will be paused without noticing. None of that is fixable — so put the effort into making a wrong guess survivable: fence the writes, decide by majority, and never let a timestamp decide who wins.

Concept sketch — the whole competency on one page

Three named groups, in the order the story runs: **SHARE** → **SPREAD** → **DOUBT**. Every node carries a question. Cover the answers, walk the map, and each answer hands you the next question.

Legend: SHARE — versions must read each other SPREAD — copy it, or split it DOUBT — what you cannot know

reversed = the one node to remember Q = retrieval cue

GROUP 1 - SHARE . data outlives code

```
-----
rolling upgrade -> old and new code run at the same time
|
+--> BACKWARD new code reads old data (easy)
+--> FORWARD old code reads new data (hard: the
|           reader is already deployed and cannot be taught,
|           so the FORMAT must let it skip AND preserve)
+--> protobuf: tags, never reused . avro: defaults, match by name
|
v
```

GROUP 2 - SPREAD . one machine becomes many

```
-----
REPLICATION - same data, many machines
one leader    -> ordering decided in one place, strong
                guarantees possible; async = stale reads
                + writes lost on failover
many leaders  -> conflicts; no uniqueness, no "not negative"
no leader     -> quorum w + r > n; no failover at all
|
SHARDING - different data, many machines
key range    -> range scans, but clustered writes = HOT SHARD
hash         -> even spread, but no range scans
hot KEY      -> neither helps; isolate or split (writes only)
|
v
```

GROUP 3 - DOUBT . the floor under all of it

```
-----
no reply is consistent with SIX different worlds
a timeout is a decision, never a discovery
clocks drift, jump backward, and fail SILENTLY
your own process can pause for a minute unnoticed
|
+--> so: decide by MAJORITY (there cannot be two)
+--> and: FENCE the writes, because zombies cannot
        be prevented -- only made harmless
```

The question per node — cover the right column

GROUP 1 — SHARE

NODE	QUESTION	ANSWER IN ONE LINE
Data outlives code	What does it actually mean for a deployment?	You can replace every running process in minutes, but five-year-old records stay in their original encoding unless you rewrite them — so old formats persist indefinitely.
Backward compatibility	State it without the word “backward”.	Newer code can read data written by older code. Usually easy, because the new code's author knew the old format existed.
Reversed: forward compatibility	Why is this the node to remember?	Older code reading newer data. Every rolling upgrade forces it, and it can't be fixed by deploying, because the reader is already running — so the format must carry the solution.
Field tags vs field names	What can you change freely in each format?	Protocol Buffers: rename freely, never reuse a tag. Avro: matches by name, so add or remove only fields that have a default.
RPC's flaw	What outcome does a network call have that a local call cannot?	Returning nothing at all, so you don't know whether it happened — which is why retries need idempotence.

GROUP 2 — SPREAD

NODE	QUESTION	ANSWER IN ONE LINE
Replication vs sharding	One line each.	Replication keeps the same data on many machines; sharding puts different data on different machines. Almost always combined.
Replication vs backup	Why isn't one a substitute for the other?	Replication propagates your accidental deletion faithfully. Backups store old snapshots so you can go back in time.
The three lag anomalies	Name them and their fixes.	Read-your-writes (read from the leader, or track the user's last-write timestamp); monotonic reads (pin the user to one replica); consistent prefix (keep causally related writes together).
Multi-leader's hard limit	What can it never enforce, and is that the product's fault?	Uniqueness and threshold constraints like a non-negative balance — two leaders can each accept a write that is individually fine. It's a fundamental limitation of distributed systems.
Quorums	Give the inequality and one reason it isn't a guarantee.	$w + r > n$ makes the read and write sets overlap. It still fails under concurrent reads and writes, partially-succeeded writes, restored-from-backup nodes, and rebalancing.
Hot shard vs hot key	Which does hashing fix?	Only the hot shard. A hot key is a workload property — hashing distributes keys uniformly, not load.
Hash mod N	Why is it the one scheme never to use?	Changing the node count moves most keys, for no benefit. Use a fixed shard count or hash ranges instead.

GROUP 3 — DOUBT

NODE	QUESTION	ANSWER IN ONE LINE
Partial failure	What makes it different from a single-machine fault?	Nondeterminism: parts break while others work, and the same operation may succeed or fail unpredictably — you may not even know which.
The six worlds	What can a missing reply mean?	Request lost, request queued, node dead, node paused, reply lost, reply delayed — indistinguishable from where you stand.

Timeout	What is it, precisely?	A decision to stop waiting and act on a guess. Never a measurement, because delay is unbounded.
Time-of-day vs monotonic	Which for what, and why?	Monotonic for durations — it can't jump backward, but its absolute value is meaningless and incomparable across machines. Time-of-day for calendar time, and it can be reset.
Timestamp ordering	What goes wrong under 3 ms of skew?	A causally later write can carry an earlier timestamp; last-write-wins then drops it silently, losing the increment.
Reversed: the zombie	Why can't you prevent it, and what do you do instead?	Pauses and delayed messages are always possible. Make its writes harmless with fencing tokens, and decide by quorum.
The majority	Why is it safe?	There cannot be two conflicting majorities at once, and it tolerates a minority failing — one of three, two of five.

Master 2x2 — the decision card you can carry to other problems

This one isn't about databases, which is the point. Two questions decide the character of any system with more than one participant — a company, a federation, a family, a fleet of machines.

COLUMNS: MUST THE HOLDERS AGREE BEFORE ANYONE CAN ACT? →

	Yes, agree first coordination before action	No, reconcile later act locally, sort it out after
ONE place holds the truth	Simple and constrained One leader, one decision-maker, one master copy. Ordering is decided for you, so strong guarantees are cheap. <i>Move:</i> take this whenever you can afford it. Its costs are honest and visible — a bottleneck, a single point of failure, and everyone waiting on one place.	Fast but stale One authority, but readers work from copies that lag. Asynchronous followers; a team acting on last week's brief. <i>Move:</i> name the anomalies you've accepted rather than discovering them in production — someone will act on information that is already out of date, including their own.
MANY places hold the truth	Correct but blocking Quorums, consensus, sign-off by committee. Correct, and it stops entirely when enough participants are unreachable. <i>Move:</i> use it for the few decisions that genuinely must not diverge, and keep the quorum small — waiting on more participants raises the odds of waiting on a slow one.	Available, and divergent Multi-leader, offline-first, autonomous regional offices, everyone editing their own copy. <i>Move:</i> most systems end up here <i>without choosing it</i> . Conflict resolution is now your job, and certain guarantees — uniqueness, “never below zero” — are simply off the table. Decide deliberately, and design the merge before you need it.

The move that matters: you can rarely change how many places hold the truth — geography, org charts and offline devices decide that for you. You can almost always choose the column, and choosing “agree first” for everything is how systems and organisations grind to a halt. Pick the handful of decisions that must not diverge; let the rest reconcile.

THE THUMB RULE, IN ONE SENTENCE

Decide where the truth lives and whether it must agree right now — every other property is a consequence of those two answers.

It travels well. A company with regional offices choosing between head-office sign-off and local autonomy is picking a column. A shared document that works offline has already picked the bottom-right and owes you a merge strategy. A hospital's records across sites, a bank's ledger across branches, a family calendar across phones — all the same two questions, and in each case the interesting failures are the ones people didn't realise they had chosen.

Symptom → diagnosis → move

Cover the middle and right columns and work down the left.

Encoding and replication

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
Fields written by upgraded instances vanish after an old instance touches the record.	A forward-compatibility break — the old reader dropped what it didn't recognise.	The format must let old readers skip <i>and preserve</i> unknown fields. New tags in Protocol Buffers; defaults in Avro.
Old records fail to decode after a release.	Backward-compatibility break — usually a removed or renamed field.	Restore the default, or the reader-schema alias. Check compatibility in a registry <i>before</i> deploying.
Two different meanings for the same field number.	A reused Protocol Buffers tag.	Never reuse tags; reserve retired ones in the schema.
A retry charged the customer twice.	A network call that timed out with an unknown outcome, retried without idempotence.	Build deduplication into the protocol. Assume every remote call may have succeeded invisibly.
User posts something, reloads, and it's gone.	Read-after-write violation on a lagging follower.	Read anything the user may have modified from the leader, or track their last-write timestamp and serve only caught-up replicas.
Content appears, disappears, reappears on refresh.	Monotonic reads violation — successive reads hitting replicas with different lag.	Pin each user to one replica, chosen by hash of user ID rather than at random.
An answer appears before the question.	Consistent prefix violation across shards.	Keep causally related writes on the same shard, or track causality explicitly.
Confirmed writes vanished after a failover.	Asynchronous replication — unreplicated writes were discarded on promotion.	Promote the most up-to-date follower: the synchronous one, or the highest log sequence number. Consider semisynchronous replication.
Two nodes both accepting writes as leader.	Split brain.	Fencing. And be aware that shutdown-based mechanisms can disable both nodes, or act after corruption.
A leader-and-follower pair can't be upgraded independently.	WAL shipping — the replication log is coupled to the storage engine's format.	Move to a logical (row-based) replication log, which also enables change data capture.
Followers diverge subtly from the leader.	Statement-based replication meeting nondeterminism — <code>NOW()</code> , <code>RAND()</code> , autoincrements, triggers.	Row-based logical replication, or make statements strictly deterministic.

Sharding and distributed reality

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
One node saturated, the rest idle.	A hot shard from skew — often a sequential partition key.	Compound the key so something spreading comes first; or split the shard; or isolate it.
Still hot after re-sharding and rehashing.	A hot <i>key</i> , not a hot shard.	Isolate it, possibly on its own machine, or add a random suffix — knowing that only writes are split, not reads.
Adding a node moved almost all the data.	Hash mod N over the node count.	Fixed shard count with many shards per node, or hash-range sharding that splits on demand.
You need more nodes than you have shards.	A fixed shard count chosen too low.	An expensive resharding. Next time choose a number divisible by many factors, or use hash ranges.

Range queries got slow after a sharding change.	Hashing destroyed the ordering the scans relied on.	Make only the first column the partition key and range-scan the later ones, so scans stay within a shard.
The cluster rebalanced during a spike and the outage worsened.	Automatic rebalancing plus automatic failure detection — an overloaded node judged dead.	Put a human in the loop; rebalance pre-emptively before known events instead.
A node was declared dead but was healthy.	Timeout too short, or an asymmetric fault where its outgoing messages were dropped.	Decide by quorum, and fence — a node must obey a majority verdict even feeling perfectly well.
A returning node corrupted data.	A zombie acting on an expired lease after a pause.	Fencing tokens on every write, rejected below the highest seen. Write immediately on acquiring a lease.
Some writes silently never take effect.	Last-write-wins with clock skew — a lagging clock cannot overwrite a faster node's value.	Stop ordering by wall clock. Track causality with version vectors.
Timeouts and odd behaviour with no obvious cause.	Clock drift going unnoticed, or a limping node passing health checks.	Monitor clock offsets and evict drifting nodes. Treat gray failure as its own case, not as “up”.

Numbers worth remembering

NUMBER	WHAT IT IS	WHY IT MATTERS IN AN ARGUMENT
81 / 66 / 33 / 32 bytes	The same record as textual JSON, binary JSON, Protocol Buffers, and Avro.	Binary JSON saves almost nothing for a real loss of readability; the schema-based formats halve it. That's the actual case for a schema.
w + r > n	The quorum condition.	The one inequality in this competency. n=3, w=2, r=2 tolerates one node down; n=5, w=3, r=3 tolerates two.
4 of 7, 5 of 9	The practical ceiling on quorum sizes.	Bigger quorums lower staleness but raise the chance of waiting on a slow replica. Past this, you're buying latency.
~30 seconds	A typical failover detection timeout.	Anchors the trade: longer means slow recovery, shorter means unnecessary failovers that make a struggling system worse.
1,000 shards / 10 nodes	The fixed-shard-count scheme: 100 shards per node.	Shows why only whole shards move on rebalance — and why you can never have more nodes than shards.
10 GB	HBase's default shard-split trigger.	Makes “shards split automatically” concrete, and shows splits can also be triggered by write throughput alone.
16 and 256	Hash ranges per node — Cassandra's default and ScyllaDB's.	Several ranges per node is how random boundaries even out into a fair share.
100 keys, writes only	Adding two random digits to a hot key.	The honest half of the trick: reads must now query all 100 and the read volume is unchanged.
~12 per month	Network faults in a medium-sized datacentre — half isolating one machine, half a whole rack.	Ends “our network is reliable”. Redundant gear helps less than expected, because it doesn't prevent human error.
200 ppm	Google's assumed clock drift: ~6 ms if resynchronised every 30 s, ~17 s if resynchronised daily.	Puts a number on how far clocks separate — and 17 seconds is far larger than most timestamp-ordering schemes assume.
~35 ms	The minimum achievable NTP error over the internet, with spikes near a second.	Anyone proposing to order events by wall clock needs to hear this first.
42.004 vs 42.003	The causally-later write carrying the earlier timestamp, under 3 ms of skew.	The concrete demonstration that last-write-wins loses data even when clocks are better than you'd expect.
100 microseconds	MiFID II's clock accuracy obligation for high-frequency trading.	Proof that good clock sync is buyable — with GPS, atomic clocks and PTP — if you actually need it.
More than two-thirds	The supermajority most Byzantine-fault-tolerant algorithms need.	Why Byzantine tolerance is impractical in ordinary datacentres, and why it can't protect you from a bug deployed to every node.

Glossary — the terms you should be able to define cold

TERM	DEFINITION, AND THE LINE THAT SEPARATES IT FROM ITS NEIGHBOUR
Backward / forward compatibility	Newer code reads older data / older code reads newer data. Always name the reader; the second is the hard one because the reader is already deployed.
Schema evolution	Changing a schema while old data stays readable. Gives schemaless flexibility with better guarantees and tooling.
Field tag vs field name	Protocol Buffers identifies fields by number, so renames are free and tags are permanent. Avro matches by name, so defaults are what make add and remove safe.
Writer's / reader's schema	Avro decodes with both: the writer's must be identical to the encoder's; the reader's may be older or newer.

Replication vs sharding	Same data on many machines / different data on different machines. Nearly always used together.
Replication log	Statement-based (breaks on nondeterminism), WAL shipping (couples the versions), or logical row-based (decoupled, and the basis of change data capture).
Eventual consistency	Stop writing and the replicas converge. “Eventually” is deliberately vague — there is no bound on lag.
Read-after-write / monotonic reads / consistent prefix	See your own writes / never see time move backward / never see an effect before its cause.
Split brain	Two nodes both believing they lead. Handled by fencing, not by hoping.
Quorum	$w + r > n$ so the read and write sets overlap. Need not be a majority; not an absolute guarantee.
Read repair / hinted handoff / anti-entropy	Three leaderless catch-up mechanisms: fix on read, hold writes for an absent replica, or reconcile in the background.
Sloppy quorum	Accepting writes on any reachable replica when the usual ones are unreachable. Availability at the cost of the overlap guarantee.
Partition key	The value deciding a record's shard. Chosen early, changed painfully.
Skew / hot shard / hot key	Unfair distribution / where it lands / the single key causing it. Hashing fixes the second, never the third.
Consistent hashing	Keys stay put when the shard count changes. Nothing to do with replica or ACID consistency.
Partial failure	Some parts broken, others fine, nondeterministically. The defining difficulty of distributed systems.
Time-of-day vs monotonic clock	Calendar time, resettable and comparable across machines / always-forward, meaningless in absolute terms, for durations only.
Gray failure	A limping node: still answering health checks, too slow to do real work. Harder to handle than a clean crash.
Lease / zombie / fencing token	A lock that expires / a former holder that doesn't know / the increasing number that makes the zombie's writes rejectable.
Byzantine fault	A node that lies. Elsewhere we assume nodes are unreliable but honest.
System model	Your stated assumptions. Partially synchronous with crash-recovery faults is the useful one for real systems.

Answer key

PART 1 · Q1 — THE VANISHING DISCOUNT CODE

(a) **Forward compatibility** broke. Stated without the word: *older code could not correctly read data written by newer code* — specifically, instances not yet upgraded encountered a field that did not exist when they were written.

(b) Because reading alone is harmless here; the damage happens on **rewrite**. An old instance reads the record, doesn't recognise `discount_code`, drops it from its in-memory representation, and then writes the record back **without the field**. Orders that were created and never touched again keep the field, because no old instance ever rewrote them. That asymmetry — only modified records lose data — is the fingerprint of this fault.

(c) The encoding must let an old reader **skip an unrecognised field and preserve it** when writing the record back. In Protocol Buffers the datatype annotation on each field tells the parser how many bytes to skip, which is what makes preservation possible; in Avro the equivalent discipline is that fields are added with defaults and resolved by name. Patching the old code is not the fix because **the old code is already deployed and running** — that is what makes this the hard direction. Even if you patch it now, the whole point of the property is that it must hold for readers you cannot reach mid-upgrade.

PART 1 · Q2 — THE NIGHTLY RELATIONAL DUMP

(a) **Avro**, and the structural property is that its schema contains **no tag numbers** — fields are identified by name. That is precisely why it is friendlier to **dynamically generated schemas**: you can generate an Avro schema from the relational schema on every run, one record per table and one field per column, and the export process need pay no attention to the schema having changed.

(b) The reader resolves the **writer's schema against the reader's schema**, matching fields **by name**. A field present in the writer's schema but absent from the reader's is ignored; a field the reader expects but the writer's schema lacks is filled from the **default declared in the reader's schema**. Field order does not matter. With Protocol Buffers this same job would require an administrator to maintain the column-to-tag mapping by hand and be very careful never to reuse a tag — exactly the failure they have already hit twice.

(c) The new obligation is conveying the writer's schema, since Avro-encoded data identifies neither field names nor types and is undecodable without it. For this use case — **a large file containing many records all encoded with the same schema** — the right mechanism is an **Avro object container file**, which includes the schema once at the beginning of the file. (Credit for naming the other two mechanisms and rejecting them: a per-record version number plus a schema registry suits a database of individually-written records; connection-time negotiation suits two processes on a network connection.)

PART 2 · Q1 — THE SOCIAL APP WITH SIX FOLLOWERS

(a) Posts vanishing after you create them is a **read-after-write (read-your-writes) violation**. A comment appearing, disappearing and reappearing is a **monotonic reads violation** — successive reads are served by replicas with different lag, so the user sees time move backward. The forty seconds lost at failover is **discarded unreplicated writes** under asynchronous replication.

(b) For read-after-write: read anything the user may have modified **from the leader** — for a social app the simple rule is own profile and own posts from the leader, everyone else's from a follower. If too much is user-editable, read from the leader for a period after each write, or have the client remember the timestamp of its last write and serve it only replicas that have caught up past it. For monotonic reads: make each user always read from **the same replica**, chosen deterministically, for example by a hash of the user ID rather than at random. Pinning does **not** solve read-after-write, because the replica a user is pinned to can still lag behind their own most recent write — consistency of *which* replica is not the same as freshness of *that* replica.

(c) At failover a follower is promoted; with asynchronous replication it may not have received the old leader's most recent writes, and the standard resolution is to **discard** them — so writes confirmed to clients turn out not to have been durable. The decision that limits the damage is **choosing the most up-to-date follower**: with semisynchronous replication, the follower the leader waited on; with asynchronous, the one with the highest log sequence number. Losing a fraction of a second may be tolerable; promoting a replica days behind would be catastrophic. Credit for naming the GitHub incident's shape — a lagging promoted leader reusing autoincrement primary keys, causing inconsistency with an external store and disclosing private data to the wrong users.

PART 2 · Q2 — THREE REGIONAL LEADERS AND TWO CONSTRAINTS

(a) It is **multi-leader replication** with asynchronous replication between regions. Neither constraint is enforceable because each leader validates against **only its own local state** at the time of the write. Two users can register the same username in two regions simultaneously: each write passes the uniqueness check locally, both succeed, and the conflict only surfaces when the writes replicate — after both clients have been told they succeeded. The balance case is the same shape: two withdrawals in different regions can each leave a non-negative balance individually while together taking the account below zero. The constraint spans writes that were never evaluated together.

(b) **No** — this is not a product limitation. It is a **fundamental limitation of distributed systems**: if two nodes can accept writes without coordinating, no amount of engineering in the database will let them enforce a constraint that depends on both writes being seen together. Switching vendors will not help. Credit for noting that multi-leader is often a retrofitted feature, so autoincrementing keys, triggers and integrity constraints also interact badly — but that is a separate, weaker point.

(c) Two structural options. Either **use a single leader** for the data carrying these constraints, accepting cross-region write latency for those operations — possibly while keeping other data multi-leader. Or **drop the constraints as hard guarantees** and handle violations after the fact: detect duplicate usernames and force a rename, allow the balance to go negative and reconcile. The topology hazard to know regardless: in an **all-to-all** topology, replication messages can **overtake one another**, so a replica may receive an update to a row before the insert that created it — a causality problem that timestamps cannot fix and that needs version vectors. I would take the single-leader option for the constrained data: the constraints are stated as product requirements, and the honest cost is latency on those specific writes rather than a class of bug that surfaces as duplicate accounts and negative balances.

PART 3 · Q1 — PINGS KEYED BY TIMESTAMP

(a) With key-range sharding on a timestamp, shards correspond to contiguous ranges of time. All **current** writes therefore land in the shard owning the present range, while every other shard holds only historical data and takes no write traffic. The **key is at fault, not the scheme**: key-range sharding is behaving exactly as designed, and it is the monotonically increasing partition key that concentrates the writes.

(b) Hashing the timestamp would spread the writes evenly, but it would **destroy range scans over the partition key**, because keys that are adjacent in time would be scattered across every shard. Their main query is “all pings in the last hour” — precisely a range scan over the partition key — so the proposed fix would turn their single most important query into a scatter-gather across all twelve nodes. It trades a hot shard for a slow system.

(c) Make the partition key **compound**, with something spreading first and the timestamp second — vehicle ID then timestamp. Writes then distribute across shards by vehicle while remaining sorted by time *within* each vehicle. The cost is that a query spanning all vehicles for a time range now needs a **separate range query per vehicle**, whose results must be combined. Credit for noting the general principle: keep the partition key's first component spreading and range-scan the later components, so scans stay within a shard.

PART 3 · Q2 — THE INSTITUTIONAL BUYER AND THE SELF-HEALING CLUSTER

(a) Because this is a **hot key**, not a hot shard. All that buyer's orders share one `buyer_id`, so they hash to one value and land in one shard no matter how many shards exist. Raising the count from 256 to 4,096 subdivides everything *except* the thing causing the problem — and it costs an expensive resharding, since every shard must be split and rewritten. Consistent hashing distributes **keys** uniformly, which is not the same as distributing **load** uniformly.

(b) Either **isolate** the hot key — a range-based scheme can put an individual hot key in a shard by itself, potentially on a dedicated machine — or **split it at the application level** by appending a random suffix, so two digits spread the writes across 100 keys. Be precise about what the second buys: it splits the **write** load only. Reads must now query all 100 keys and combine the results, so the **read volume against that data is unchanged**. It also needs bookkeeping to record which keys are split, and a process for promoting a normal key to a split one, since applying it to every key would be pure overhead.

(c) The incident is **automatic rebalancing interacting with automatic failure detection**. A node was overloaded by the spike and became slow; its peers concluded it was dead and rebalanced its shards away; moving that data added load to the network and the remaining nodes, making *them* slow, and risking further false suspicions — a cascading failure caused entirely by the automation. I would put a **human in the loop** for rebalancing, accepting that it is slower in exchange for avoiding operational surprises, and **rebalance preemptively** ahead of known traffic events rather than reactively during them.

PART 4 · Q1 — THE PAUSED WORKER AND THE CORRUPTED FILE

(a) The first worker became a **zombie** — a former leaseholder that has not yet discovered it lost the lease and continues acting as though it still holds it, with no awareness that time has passed. The proposed fix cannot eliminate the problem because it only makes the window smaller, never zero: garbage collection pauses can be reduced but not abolished, and the same failure arises with **no pause at all** if a write request is merely **delayed in the network** and arrives after the lease has been reassigned — packets can be delayed by a minute or more. Any fix based on making the timing better is fighting an unbounded quantity.

(b) **Fencing tokens.** The lock service returns a number with every grant that **increases each time a lease is issued**. Every write to the storage service must carry the client's current token, and the storage service **remembers the highest token it has processed and rejects any write carrying a lower one**. So worker 1 holds token 33, pauses, and worker 2 acquires 34 and writes; when worker 1 wakes and writes with 33, the service rejects it because it has already accepted 34. On acquiring a lease a client should **immediately make a write**, because once that write completes any earlier zombie is fenced off. Credit for noting that a conditional-write or compare-and-set facility — S3's conditional writes, Azure's conditional headers, Google Cloud Storage's request preconditions — achieves the same effect for a single storage service.

(c) The popular alternative is **shutting the zombie down** — disconnecting it from the network, stopping the VM, or powering the machine off, sometimes called STONITH. It is ineffective for two reasons: it **does not protect against a delayed write already in flight** from a client that has since crashed, and **by the time the zombie is detected and shut down the damage may already be done**. A third acceptable answer: badly designed shutdown mechanisms can result in all the nodes shutting one another down.

PART 4 · Q2 — WRITES THAT NEVER TAKE EFFECT

(a) Under **last-write-wins** the replica keeps whichever value carries the greatest timestamp and discards the rest. A node whose clock lags is therefore **unable to overwrite a value previously written by a node with a faster clock** until the skew between them has elapsed. The write is accepted and acknowledged, then silently discarded during conflict resolution — no error is reported to the application, which is why it looks like the write “never took effect”. Sub-5 ms skew does not make this safe because **safety here is not a function of how small the skew is**: the worked example loses an increment with skew under 3 ms, since what matters is whether the causally-later write happens to receive a smaller timestamp, and any positive skew makes that possible. Correlating with one region is the expected fingerprint of one group of clocks running slightly slow.

(b) The second defect is that last-write-wins **cannot distinguish genuinely concurrent writes from sequential ones** — writes that occurred in quick succession, where the second causally depends on the first, are treated identically to writes where neither writer knew about the other. The mechanism that addresses it is **version vectors**, which track causality explicitly rather than inferring it from time. (A third defect worth credit: two nodes can generate the same timestamp at millisecond resolution, needing a tiebreaker that can itself violate causality.)

(c) Because **clock faults fail silently**. A defective CPU or a misconfigured network breaks loudly and gets fixed quickly; a drifting quartz clock or a misconfigured NTP client lets everything appear to work while diverging further from reality, so the result is **silent, subtle data loss rather than a dramatic crash**. Nobody is paged, nothing appears in a dashboard, and the loss is discovered — if ever — long afterwards by a user. The operational implication is that you must **monitor clock offsets across the cluster and evict any node whose clock drifts too far**, because the system will not tell you.

HOW TO SCORE YOURSELF

Two marks, both quick. **Completeness:** did you name every element, and — for this competency especially — did you get the *direction* right? Which code is reading, which node accepts writes, which key is hot, who believes what about whom. Direction errors are the characteristic failure here and matter more than a missing term. **Phrasing:** could you have said it in a design review without hand-waving? If you wrote “the old version broke” where the key says “an old reader dropped the unrecognised field when it rewrote the record”, the idea is there and the mechanism isn't — and the mechanism is what changes anyone's mind.

Spaced-review tracker

Review at **day 1, 3, 7, 16, 35**. Write the date in the box when you pass. On a fail, reset that row to a 1-day interval and start again.

ITEM	DAY 1	DAY 3	DAY 7	DAY 16	DAY 35	FAILS
1 • Backward vs forward compatibility — stated without the words						
2 • Why forward compatibility can't be fixed by deploying						
3 • Protobuf tags vs Avro names — what each lets you change						
4 • How a reader obtains the writer's schema — three contexts						
5 • Why a remote call is not a local call — four differences						
6 • Single vs multi vs leaderless — who accepts writes						
7 • Synchronous, asynchronous, semisynchronous — and their costs						
8 • The three replication log implementations and their traps						
9 • The three lag anomalies and their fixes						
10 • Failover: what goes wrong, and the one decision that matters						
11 • What multi-leader structurally cannot enforce						
12 • Quorums: $w + r > n$, and why it isn't a guarantee						
13 • Read repair, hinted handoff, anti-entropy						
14 • Replication is not backup						
15 • Key-range vs hash sharding — what each buys and breaks						
16 • Hot shard vs hot key — which hashing fixes						
17 • The three hash-to-shard schemes, and why not mod N						
18 • The random-suffix trick and exactly what it doesn't reduce						
19 • Automatic rebalancing plus failure detection = cascade						
20 • The six worlds behind a missing reply						
21 • Time-of-day vs monotonic clocks						

22 • Why last-write-wins loses data at 3 ms of skew						
23 • Zombies and fencing tokens — and why STONITH fails						
24 • The majority rule, and why it is safe						
25 • System models — timing and node failure						
26 • Master matrix — where truth lives × must it agree now						

Final quiz — no notes, twenty minutes, write the answers

1. Define backward and forward compatibility without using either word, and say which one a rolling upgrade forces on you and why it cannot be fixed by deploying a patch.
2. An old instance reads a record containing a field it has never heard of, then writes the record back. What must the encoding guarantee, and what is lost if it doesn't?
3. Name the three replication log implementations, and give the specific defect of each that the others avoid.
4. A user posts a comment and it vanishes on reload; another user sees a comment appear, vanish and reappear. Name both anomalies and give the fix for each, then say why one fix doesn't solve the other.
5. State the quorum condition, work out what $n=5$, $w=3$, $r=3$ tolerates, and give two reasons the condition can hold while a read still returns a stale value.
6. Why can a multi-leader system not guarantee unique usernames? Is this the database's fault?
7. A table sharded by timestamp has one saturated node. Explain the mechanism, say whether the key or the scheme is at fault, and give the fix along with the query cost it imposes.
8. Explain why hashing fixes a hot shard but not a hot key, and state precisely what the random-suffix workaround does and does not reduce.
9. You send a request and receive no reply. List at least four distinct things that may have happened, and say what a longer timeout would tell you about which one it was.
10. A client pauses for two minutes, its lease expires, another client takes over, and the first wakes up and writes. Name what it has become, why you cannot prevent it, and the mechanism that makes its write harmless.